

Rapport DevOps — BankApp Spring Boot + React

Page de garde

Module : DevOps

Projet : BankApp — Application de gestion bancaire avec CI/CD

Étudiant : El-Mehdi Taoufik

Année universitaire : 2025-2026

Dépôt GitHub : <https://github.com/Hasanpiker5543/spring-bank-project>

1. Introduction

Ce rapport présente la réalisation d'une application web de gestion bancaire développée avec une architecture moderne séparant le backend et le frontend. Le projet a ensuite été intégré dans une chaîne DevOps complète comprenant la gestion de version avec Git, la conteneurisation avec Docker, l'orchestration locale avec Docker Compose et l'intégration continue avec GitHub Actions.

L'objectif principal du projet est de mettre en pratique les notions du module DevOps à travers une application concrète. L'application permet de gérer des clients, des comptes bancaires, des transactions, des transferts d'argent, ainsi que l'authentification des utilisateurs.

2. Présentation du sujet choisi

Le sujet choisi est une application de gestion bancaire. Cette application permet à un administrateur de gérer les clients et les comptes, et permet aux utilisateurs connectés de consulter leurs informations bancaires et d'effectuer des transferts.

Fonctionnalités principales

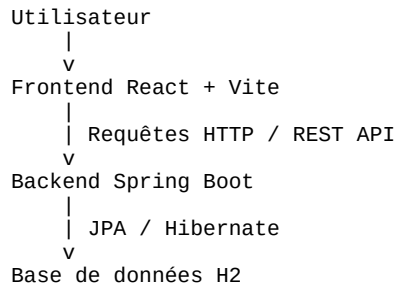
- Authentification : inscription, connexion et déconnexion.
- Gestion des clients.
- Gestion des comptes bancaires.
- Consultation des transactions récentes.
- Transfert d'argent entre comptes.
- Recherche d'un destinataire par numéro de compte, téléphone ou nom d'utilisateur.
- Tableau de bord administrateur.
- Tableau de bord client avec visibilité limitée à ses propres données.

3. Architecture de l'application

L'application est composée de deux parties principales :

- Un backend développé avec Spring Boot.
- Un frontend développé avec React et Vite.

Schéma global



Structure du projet

```
spring-bank-project/
├── backend/
│   ├── src/main/java/com/bankapp/
│   │   ├── config/
│   │   ├── controller/
│   │   ├── dto/
│   │   ├── model/
│   │   ├── repository/
│   │   ├── service/
│   │   └── BankApplication.java
│   ├── src/test/java/com/bankapp/
│   ├── Dockerfile
│   └── pom.xml
├── frontend/
│   ├── src/
│   │   ├── services/
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── styles.css
│   ├── Dockerfile
│   ├── nginx.conf
│   └── package.json
├── .github/workflows/ci-cd.yml
├── docker-compose.yml
└── README.md
```

4. Choix techniques

Backend

Le backend est développé avec Spring Boot 3. Ce choix permet de créer rapidement une API REST robuste, structurée et facilement maintenable. Spring Data JPA est utilisé pour gérer la persistance des données avec Hibernate.

Technologies backend :

- Java 21
- Spring Boot

- Spring Web
- Spring Data JPA
- H2 Database
- Maven

Frontend

Le frontend est développé avec React. Vite est utilisé pour accélérer le développement et la construction de l'application.

Technologies frontend :

- React
- Vite
- Axios
- Lucide React
- CSS moderne

Base de données

La base de données utilisée est H2 en mémoire. Elle est adaptée pour le développement, les tests et la démonstration du projet.

5. Modèle de données

Client

Le modèle `Client` représente un client de la banque.

Attributs principaux :

- `id`
- `username`
- `phone`

Account

Le modèle `Account` représente un compte bancaire.

Attributs principaux :

- `id`
- `accountNumber`
- `balance`

- `client`

Transaction

Le modèle `Transaction` représente une opération bancaire.

Attributs principaux :

- `id`
- `type`
- `amount`
- `date`
- `account`

AppUser

Le modèle `AppUser` représente un utilisateur de l'application.

Attributs principaux :

- `id`
- `username`
- `password`
- `role`
- `client`

6. API REST

Le backend expose plusieurs endpoints REST.

Authentication

```
POST /api/auth/login
POST /api/auth/register
POST /api/auth/logout
```

Clients

```
GET    /api/clients
POST   /api/clients
PUT    /api/clients/{id}
DELETE /api/clients/{id}
```

Comptes

```
GET    /api/accounts
POST   /api/accounts
PUT    /api/accounts/{id}
DELETE /api/accounts/{id}
```

Transactions

```
GET /api/transactions
GET /api/transactions/recent
GET /api/transactions/account/{accountId}
```

Transfert

```
POST /api/transfers
```

Exemple de requête de transfert :

```
{
  "fromAccountId": 1,
  "toAccountNumber": "ACC-MEHDI-001",
  "amount": 100
}
```

7. Mise en œuvre de l'application

Le backend suit une architecture en couches :

- ``controller`` : expose les endpoints REST.
- ``service`` : contient la logique métier.
- ``repository`` : communique avec la base de données.
- ``model`` : représente les entités JPA.
- ``dto`` : contient les objets de transfert de données.

Le frontend communique avec le backend à l'aide d'Axios. Les appels API sont centralisés dans le fichier ``src/services/api.js``.

L'interface utilisateur propose une page de connexion, une page d'inscription, un tableau de bord, un formulaire de transfert, un formulaire d'ajout de client et un formulaire d'ajout de compte pour l'administrateur.

8. Stratégie Git

Le projet est versionné avec Git et hébergé sur GitHub.

Dépôt GitHub

<https://github.com/Hasanpiker5543/spring-bank-project>

Branches recommandées

- ``main`` : branche principale stable.
- ``develop`` : branche d'intégration.

- `feature/\*` : branches pour les nouvelles fonctionnalités.

Exemples de workflow

```
feature/authentication -> develop -> main
feature/docker -> develop -> main
feature/ci-cd -> develop -> main
```

Les commits utilisent des messages clairs décrivant les changements effectués, par exemple :

```
Add Docker and CI CD setup
Add author name to README
```

9. Conteneurisation avec Docker

Le projet contient deux Dockerfiles :

- `backend/Dockerfile`
- `frontend/Dockerfile`

Backend Dockerfile

Le backend utilise une image Maven avec Java 21 pour construire le fichier JAR, puis une image Java Runtime pour exécuter l'application.

Étapes principales :

- Copier le code backend.
- Construire le projet avec Maven.
- Copier le JAR dans une image légère.
- Exposer le port `8080`.

Frontend Dockerfile

Le frontend utilise Node.js pour construire l'application React, puis Nginx pour servir les fichiers statiques.

Étapes principales :

- Installer les dépendances avec `npm ci`.
- Construire l'application avec `npm run build`.
- Servir le dossier `dist` avec Nginx.
- Exposer le port `80` dans le conteneur.

10. Docker Compose

Le fichier `docker-compose.yml` permet de lancer toute l'application avec une seule commande.

Commande :

```
docker compose up --build
```

Services :

- `backend` : application Spring Boot accessible sur `http://localhost:8080`.
- `frontend` : interface React servie par Nginx accessible sur `http://localhost:5173`.

Extrait du fichier :

```
services:
  backend:
    build:
      context: ./backend
    ports:
      - "8080:8080"

  frontend:
    build:
      context: ./frontend
    ports:
      - "5173:80"
    depends_on:
      - backend
```

11. Tests automatisés

Un test de chargement du contexte Spring Boot a été ajouté.

Fichier :

```
backend/src/test/java/com/bankapp/BankApplicationTests.java
```

Commande d'exécution :

```
cd backend
.\mvnw.cmd test
```

Résultat obtenu :

```
Tests run: 1, Failures: 0, Errors: 0
BUILD SUCCESS
```

Le frontend est validé avec la commande :

```
npm.cmd run build
```

Résultat obtenu :

```
built successfully
```

12. Pipeline CI/CD avec GitHub Actions

Le pipeline CI/CD est défini dans le fichier :

```
.github/workflows/ci-cd.yml
```

Déclenchement

Le pipeline est déclenché automatiquement lors de :

- push vers `main` ou `develop`
- pull request vers `main` ou `develop`

Étapes du pipeline

Le pipeline exécute les étapes suivantes :

1. Récupération du code source.
2. Installation de Java 21.
3. Exécution des tests backend avec Maven.
4. Installation de Node.js 22.
5. Installation des dépendances frontend avec `npm ci`.
6. Build du frontend React.
7. Build de l'image Docker backend.
8. Build de l'image Docker frontend.
9. Publication des images Docker sur GitHub Container Registry après un push réussi sur `main`.

Badge de pipeline

Le README contient un badge GitHub Actions indiquant l'état du pipeline.

```
https://github.com/Hasanpiker5543/spring-bank-project/actions/workflows/ci-cd.yml
```

13. Publication des images Docker

Les images Docker sont publiées sur GitHub Container Registry.

Images :

```
ghcr.io/Hasanpiker5543/spring-bank-backend:latest  
ghcr.io/Hasanpiker5543/spring-bank-frontend:latest
```

Cette publication est automatique lorsque le pipeline réussit sur la branche `main`.

14. Difficultés rencontrées

Installation de Maven

Au début du projet, Maven n'était pas disponible sur la machine locale. La solution a été d'utiliser Maven Wrapper afin de lancer le backend sans installation globale de Maven.

Problème de Lombok

Certaines méthodes générées par Lombok n'étaient pas reconnues correctement. La solution a été d'ajouter explicitement les getters, setters et constructeurs nécessaires.

Erreurs de transfert

Des erreurs de transfert ont été rencontrées à cause de conflits dans les identifiants de données initiales. La solution a été de réinitialiser les compteurs d'identité dans `data.sql`.

Docker non installé localement

Docker n'était pas disponible sur la machine Windows locale. Les fichiers Docker ont été préparés et le build Docker est validé dans GitHub Actions.

15. Sécurité et bonnes pratiques

- Les fichiers `.env` sont ignorés dans `.gitignore`.
- Les dossiers `node\_modules` et `target` ne sont pas versionnés.
- Le frontend limite l'affichage des données selon le rôle utilisateur.
- L'administrateur peut voir tous les clients et comptes.
- Un client normal ne voit que ses propres données.

16. Répartition du travail

Le projet a été réalisé par :

- El-Mehdi Taoufik

Les tâches réalisées comprennent :

- Développement backend Spring Boot.
- Développement frontend React.
- Mise en place de l'authentification.
- Gestion des clients, comptes, transactions et transferts.
- Création des Dockerfiles.
- Création du fichier Docker Compose.
- Mise en place du pipeline CI/CD GitHub Actions.
- Rédaction de la documentation.

17. Conclusion

Ce projet a permis de développer une application bancaire complète tout en mettant en pratique les principes DevOps. L'application est versionnée avec Git, documentée avec un README complet, conteneurisée avec Docker et intégrée dans un pipeline CI/CD avec GitHub Actions.

La chaîne DevOps permet d'automatiser les tests, la construction du frontend, la construction des images Docker et leur publication sur un registre. Cette approche améliore la qualité du projet, facilite son déploiement et rend l'application plus professionnelle.

18. Perspectives

Des améliorations possibles peuvent être ajoutées :

- Utiliser une base de données PostgreSQL avec Docker Compose.
- Ajouter plus de tests unitaires et d'intégration.
- Ajouter un linter backend et frontend.
- Ajouter un environnement de staging.
- Déployer l'application sur un serveur cloud.
- Ajouter Kubernetes avec Minikube pour obtenir le bonus.

19. Annexes

Commandes utiles

Lancer le backend :

```
cd backend
.\mvnw.cmd spring-boot:run
```

Lancer le frontend :

```
cd frontend
npm.cmd install
npm.cmd run dev
```

Lancer avec Docker :

```
docker compose up --build
```

Tester le backend :

```
cd backend
.\mvnw.cmd test
```

Construire le frontend :

```
cd frontend  
npm.cmd run build
```

Liens

- Dépôt GitHub : <https://github.com/Hasanpiker5543/spring-bank-project>
- Pipeline CI/CD : <https://github.com/Hasanpiker5543/spring-bank-project/actions/workflows/ci-cd.yml>